```
=== HAMMING CODE - ERROR DETECTION AND CORRECTION ===

Enter number of 4-bit data blocks in the stream: 2

--- Block 1 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 0 1 1
Original Data Bits            : 1011
Encoded Hamming Code (7 bits) : 0110011
Enter bit position to introduce error (1-7, 0 = no error): 3
Transmitted Code (error at pos 3): 0100011
Received Code : 0100011
Checking for errors...
Error detected at bit position: 3 -> Correcting...
Corrected Code      : 0110011
Recovered Data Bits : 1011

--- Block 2 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 0 1 1 0
Original Data Bits            : 0110
Encoded Hamming Code (7 bits) : 1100110
Enter bit position to introduce error (1-7, 0 = no error): 6
Transmitted Code (error at pos 6): 1100100
Received Code : 1100100
Checking for errors...
Error detected at bit position: 6 -> Correcting...
Corrected Code      : 1100110
Recovered Data Bits : 0110
```

Ex No: 5 ERROR DETECTION AND CORRECTION USING HAMMING CODE

Aim:

To write a program to implement error detection and correction using the Hamming code

Algorithm:

1) Start the program
2) Read the number of 4 bit data blocks (n).
3) For each data block:
   * Place data bits in positions 3, 5, 6, 7 and compute parity bits for 1, 2, 4
   * Form the 7-bit Hamming code.
4) Introduce a single bit error at the user-specified position.
5) At the receiver, compute the syndrome (s1,s2,s4)
6) Find the error position
7) If an error is detected, correct it by flipping.
8) Extract and display the original 4 data bits
9) Repeat for all n data blocks.
10) Stop the program.

Program:

```
def encode(d):
    h = [0] * 7
    h[2], h[4], h[5], h[6] = d
    h[0] = h[2] ^ h[4] ^ h[6]
    h[1] = h[2] ^ h[5] ^ h[6]
```

```
    h[3] = h[4] ^ h[5] ^ h[6]
    return h
def correct(h):
    s = (h[0]^h[2]^h[4]^h[6]) + 2*(h[1]^h[2]^
        h[5]^h[6]) + 4*(h[3]^h[4]^h[5]^h[6])
    if s:
        print("Error at bit {s}, corrected.")
        h[s-1] ^= 1
    else:
        print("No error.")
    return h
n = int(input("Number of blocks: "))
for i in range(n):
    d = list(map(int, input("Enter 4 bits: ").
        split()))
    h = encode(d)
    print("Encoded:", ''.join(map(str, h)))
    print("Received:", ''.join(map(str, h)))
    correct(h)
print("corrected:", ''.join(map(str, h)))
print("Data:", ''.join(map(str, [h[2], h[4],
        h[5], h[6]])))
```

Result:

the program to implement error detection and correction using Hamming code was successfully written in Python, executed and verified using a sample input data stream.